

Lab Session 2

Proof by Induction

Software Verification

Nova School of Science and Technology
Mário Pereira `mjp.pereira@fct.unl.pt`

Version of September 21, 2026

Preface

From this point on, we will start using some auxiliary definitions from the Rocq standard library. If you have an older version of the system, you shall start your Rocq developments by the following lines:

```
From Coq Require Import Nat. (* Library on natural numbers *) Coq
From Coq Require Import Bool.Bool. (* Library on Boolean values *)
From Coq Require Import Lia. (* Linear integer arithmetic decision
    procedure *)
From Coq Require Import Lists.List. (* Library on lists *)
Import ListNotations. (* Syntactic-sugar for list values *)
```

Otherwise, for newer versions, just replace Coq by Stdlib, as follows:

```
From Stdlib Require Import Nat. (* Library on natural numbers *) Coq
From Stdlib Require Import Bool.Bool. (* Library on Boolean values
    *)
From Stdlib Require Import Lia. (* Linear integer arithmetic
    decision procedure *)
From Stdlib Require Import Lists.List. (* Library on lists *)
Import ListNotations. (* Syntactic-sugar for list values *)
```

1 Proof by Induction

1.1 Natural Numbers

Exercise 1. Prove the following lemma:

Lemma `add_0_r` : $\forall n: \text{nat},$
 $n + 0 = n.$

Coq

□

Exercise 2. Prove the following lemma:

Lemma `mult_0_r` : $\forall n: \text{nat},$ *Coq*
 $n \times 0 = 0.$

□

Exercise 3. Prove the following lemma:

Lemma `plus_n_Sm` : $\forall n\ m: \text{nat},$ *Coq*
 $S (n + m) = n + S\ m.$

□

Exercise 4. Prove the following lemma:

Lemma `add_comm` : $\forall n\ m: \text{nat},$ *Coq*
 $n + m = m + n.$

□

Exercise 5. Prove the following lemma:

Lemma `add_assoc` : $\forall n\ m\ p: \text{nat},$ *Coq*
 $n + (m + p) = (n + m) + p.$

□

Exercise 6. Given the following definitions

`let rec power (n: nat) (m: nat) : nat =` *OCaml*
`match m with`
`| 0 => S 0`
`| S m' => mult n (power n m')`

`let rec pow (n: nat) (m: nat) (r: nat) : nat :=`
`match m with`
`| 0 => r`
`| S m' => pow n m' (r * n)`

`let power_alt (n: nat) (m: nat) : nat =`
`pow n m 1`

show that the following Lemma holds:

Lemma `power_alt_correct` : $\forall n\ m: \text{nat},$ *Coq*
 $\text{power } n\ m = \text{power_alt } n\ m.$

Hint: you might want to prove an auxiliary Lemma about the result of `pow` $p\ q$, for any natural numbers p and q .

□

Exercise 7. Provide a definition for the function `fact`, of type $\text{nat} \rightarrow \text{nat}$, which computes the factorial of its argument. □

Exercise 8. Consider the following alternative formulation of the factorial function:

```
Fixpoint fact_acc (n: nat) (a: nat) : nat :=
  match n with
  | 0 => a
  | S n' => fact_acc n' (n × a)
  end.
```

Coq

```
Definition fact_alt (n: nat) : nat :=
  fact_acc n 1.
```

Prove that `fact_alt` is a correct implementation of the factorial function, *i.e.*, prove the following statement:

```
Lemma fact_alt_correct : ∀ n: nat,
  fact_alt n = fact n.
```

Coq

□

1.2 Lists

Exercise 9. Prove the following lemma:

```
Lemma nil_app: ∀ l : list nat,
  [] ++ l = l.
```

Coq

□

Exercise 10. Prove the following lemma:

```
Lemma app_assoc : ∀ l1 l2 l3: list nat,
  (l1 ++ l2) ++ l3 = l1 ++ (l2 ++ l3).
```

Coq

□

Consider the following definition of a function that reverses a list of natural numbers:

```
Fixpoint rev (l: list nat) : list nat :=
  match l with
  | [] => []
  | h :: t => rev t ++ [h]
  end.
```

Coq

Exercise 11. Prove the following lemma:

```
Lemma app_nil_r : ∀ l : list nat,
  l ++ [] = l.
```

Coq

□

Exercise 12. Prove the following lemma:

```
Lemma rev_length : ∀ l: list nat,
  length (rev l) = length l.
```

Coq

□

Exercise 13. Prove the following lemma:

Lemma `rev_app_distr` : $\forall l1\ l2 : \text{list},$
 $\text{rev } (l1 ++ l2) = \text{rev } l2 ++ \text{rev } l1.$

Coq

□

Exercise 14. Prove the following lemma:

Lemma `rev_involutive` : $\forall l : \text{list},$
 $\text{rev } (\text{rev } l) = l.$

Coq

□

1.3 Trees

Consider the following definition for a binary tree of natural numbers:

Inductive `tree` : `Type` :=
| `Leaf` : `tree`
| `Node` (`l`: `tree`) (`v`: `nat`) (`r`: `tree`).

Coq

Exercise 15. Provide a definition for the function `mirror`, of type `tree` $\rightarrow$ `tree`, which “mirrors” its argument.

Your `mirror` check should check the following examples:

Example `mirror1` : `mirror Leaf` = `Leaf`.

Coq

Proof. `reflexivity. Qed.`

Example `mirror2` :

`mirror (Node (Node Leaf 0 Leaf) 1 (Node Leaf 2 Leaf)) =`
`Node (Node Leaf 2 Leaf) 1 (Node Leaf 0 Leaf).`

Proof. `reflexivity. Qed.`

□

Exercise 16. Prove the following lemma:

Lemma `mirror_involutive`: $\forall t : \text{tree},$
 $\text{mirror } (\text{mirror } t) = t.$

Coq

□